

Research and Architecture Handbook: RISC-V Controlled FPGA SmartNIC for 5G Edge Networks

Primary Objective Evaluation

The proposed architectural endeavor—developing a RISC-V Controlled FPGA SmartNIC for Network Slicing and Quality of Service (QoS) Enforcement—represents a highly strategic alignment of telecommunications infrastructure research and very-large-scale integration (VLSI) engineering. The analysis unequivocally determines that this project is exceptionally relevant to the mandate of Bharat5G Labs, the trajectory of 5G/6G research, the evolution of SmartNIC ecosystems, and the career progression of a silicon engineer.

Bharat5G Labs, initiated by the Department of Telecommunications (DoT), is tasked with accelerating 5G research, creating intellectual property (IP), and developing indigenous network solutions for edge and rural deployments¹. A hardware-accelerated network slicing solution directly addresses a core bottleneck in the 5G User Plane Function (UPF)⁴. By delivering a functional Field Programmable Gate Array (FPGA) prototype, the project fulfills the laboratory's core deliverables, which mandate the development of testable products, standardization contributions, and academic publications².

From a career perspective, mastering the integration of a RISC-V soft processor with a 100Gbps network data plane constitutes an elite-level VLSI and System-on-Chip (SoC) design capability. The project demands rigorous expertise in AXI-Stream protocols, hardware-software co-design, PCIe Direct Memory Access (DMA), and strict timing closure at high frequencies⁶. The semiconductor industry actively seeks engineers capable of bridging the gap between networking abstractions and cycle-accurate Register Transfer Level (RTL) implementations. Furthermore, the academic community is currently deeply engaged in exploring programmable data planes and QoS enforcement, making this a highly publishable endeavor in top-tier IEEE and ACM venues⁹.

Part 1: The SmartNIC Ecosystem

To architect a novel network interface, the foundational ecosystem must be systematically deconstructed from first principles.

Taxonomy of Network Processors

The transition from basic network connectivity to in-network computing represents a fundamental shift in data center and telecommunications architecture.

Technology	Definition and Architectural Function
------------	---------------------------------------

NIC	A standard Network Interface Card acts solely as a physical bridge between the network and the host. It provides the Ethernet Media Access Control (MAC) and physical transceiver (PHY), moving packets into host memory via DMA. The host CPU executes all protocol processing ⁷ .
SmartNIC	A NIC augmented with programmable compute elements (ARM cores, FPGAs, or custom ASICs). It offloads infrastructure tasks such as virtual switching (vSwitch), cryptography, and storage protocols from the host CPU ¹³ .
FPGA SmartNIC	A SmartNIC utilizing Field Programmable Gate Arrays to provide a reconfigurable, clock-cycle-accurate hardware data plane. It allows developers to define custom packet processing pipelines in RTL, offering deterministic latency critical for telecommunications ¹¹ .
DPU	A Data Processing Unit typically combines a hardened networking ASIC with an array of multi-core processors (e.g., ARM) on a single chip. DPUs focus on running complete Linux operating systems on the network card to isolate infrastructure management from tenant workloads ⁹ .
NPU	A Network Processing Unit is a specialized, highly parallel ASIC optimized specifically for packet processing, routing, and deep packet inspection (DPI). While fast, NPUs generally lack the general-purpose compute capabilities of DPUs ¹⁶ .
IPU	An Infrastructure Processing Unit (an Intel paradigm) physically separates cloud provider infrastructure workloads from the tenant CPU. It uses programmable pipelines

	(like P4) combined with compute complexes to handle network virtualization, storage, and security ¹⁷ .
Programmable Dataplane	An architecture where the logic dictating how packets are parsed, modified, and forwarded is defined by software (e.g., P4) rather than fixed-function silicon ¹⁹ .
In-Network Computing	The paradigm of executing application-level logic—such as machine learning inference, database query aggregation, or baseband processing—directly within network switches or SmartNICs as packets transit the network ⁹ .

Operational Rationale and Accelerated Workloads

SmartNICs exist primarily to eliminate the "datacenter tax." In modern hyperscale and 5G environments, up to 30% of a host CPU's cycles can be consumed by processing Open vSwitch (OVS) rules, managing TCP/IP stacks, enforcing security policies, and handling hypervisor traffic routing¹³. SmartNICs solve this by intercepting and processing this traffic inline, freeing the expensive host CPU exclusively for revenue-generating application workloads¹³. Workloads accelerated by these devices include network virtualization (VXLAN/Geneve encapsulation), storage virtualization (NVMe over Fabrics), cryptography (IPsec/TLS offload), and 5G User Plane Function (UPF) routing⁵.

Platform Architecture Analysis

An exhaustive analysis of prevailing commercial and open-source platforms highlights the diverse architectural strategies employed in the ecosystem.

Platform	Architecture & Processing Model	Control Plane	Data Plane	Strengths	Weaknesses
NVIDIA BlueField [cite: 9, 14]	DPU combining a ConnectX ASIC with ARM cores.	Runs DOCA software stack on hardened multi-core	Fixed-function ASIC pipeline with programmability	Highly mature software ecosystem; excellent	Rigid data plane prevents deep cycle-accurate

	Uses a "look-aside" or inline processing model.	ARM processors.	ble steering.	out-of-the-box performance for RDMA/RoCE.	ate custom packet processing modifications ⁹ .
AMD Pensando [cite: 14]	DPU centered around a highly parallel, P4-programmable match-action pipeline.	Embedded ARM cores managing policies and exceptions.	P4-programmable match-action tables in silicon.	Exceptional for custom protocol parsing and edge firewalling at line rate.	Closed-source and proprietary, preventing academic or open-hardware extensions ¹⁴ .
Intel Mount Evans [cite: 17, 18]	IPU integrating up to 16 ARM Neoverse cores with a P4-programmable packet processing engine.	Dedicated management processor for secure boot and life-cycle management.	Highly programmable parser, exact/wildcard match tables capable of 200Mpps.	Massive cross-sectional bandwidth; hardware-enforced isolation ¹⁸ .	Entirely proprietary internal RTL; inaccessible for custom VLSI research.
Corundum [cite: 12, 24, 25]	Open-source FPGA SmartNIC using custom PCIe DMA and massive queue state managers in Block	Host CPU configures AXI-Lite registers and handles descriptors.	Reconfigurable RTL with modular Transmit (TX) and Receive (RX) engines.	Fine-grained hardware control over 10,000+ queues; precise TDMA scheduling ¹² .	Complex custom DMA creates a steep learning curve for integration with vendor IPs.

	RAM.				
OpenNIC [cite: 6, 27, 28]	AMD/Xilinx open-source FPGA shell separating host infrastructure from user logic boxes (250MHz/322MHz).	Host driver configures standard Xilinx QDMA registers via AXI-Lite.	Pass-through AXI-Stream interfaces terminating at custom user "roles".	Leverages highly optimized, stable vendor IPs (QDMA, CMAC); easy prototyping ⁷ .	Merely a shell; lacks native network stacks, schedulers, or routing logic out of the box ³⁰ .
NetFPGA [cite: 31, 32, 33]	Long-standing academic networking platform; modern iterations build atop OpenNIC.	Host-based control via specialized drivers and SDNet tools.	Reconfigurable logic, often targeted by P4-to-FPGA compilers.	Massive academic community support; ideal for teaching and research ³² .	Can be slow to support the absolute latest commercial high-speed protocols natively.

Part 2: 5G and 6G Relevance

The integration of SmartNICs into cellular architectures represents a critical evolution necessary to sustain the bandwidth and latency requirements of emerging standards.

SmartNIC Deployment in Cellular Topologies

In the **5G Core**, the User Plane Function (UPF) is responsible for routing all user traffic between the Radio Access Network (RAN) and the Data Network. The UPF must perform Deep Header Inspection (DHI) and Longest Prefix Match (LPM) routing on General Packet Radio Service Tunneling Protocol (GTP-U) packets⁵. Software-based UPFs running on standard servers fail to sustain 100Gbps due to memory access bottlenecks. SmartNICs are deployed here to execute hardware-accelerated routing and packet classification⁴.

Within the **Open RAN (O-RAN)** framework, the base station is disaggregated into the Radio Unit (RU), Distributed Unit (DU), and Central Unit (CU). The fronthaul connection between the RU and DU utilizes the eCPRI protocol, which mandates extreme timing precision, often requiring packets to be processed within a 1-microsecond window³⁴. SmartNICs in the DU perform inline acceleration for eCPRI parsing, Media Access Control (MAC) address flow

steering, and IEEE 1588 Precision Time Protocol (PTP) timestamping³⁵.

In **Multi-Access Edge Computing (MEC)** and **Edge Cloud** deployments, physical space and power are severely constrained. SmartNICs enable these edge servers to maintain high throughput by handling network isolation, VXLAN decapsulation, and security firewalling without requiring bulky Top-of-Rack (ToR) switches¹⁰.

For **UAV Networks** and **Drone Networks**, Size, Weight, and Power (SWaP) constraints are paramount. A highly integrated, single-chip FPGA SmartNIC provides ultra-low-latency, deterministically timed communication channels necessary for drone swarm coordination, bypassing heavy operating system overheads³⁶.

Looking toward **6G Architectures**, the boundary between computation and networking will dissolve. Terabit-per-second holographic data streams, AI-assisted routing, and sub-millisecond control loops will necessitate programmable hardware data planes, as general-purpose CPUs mathematically cannot scale to terabit switching speeds³⁷.

Bottlenecks, Network Slicing, and QoS

The primary bottlenecks in modern 5G systems are CPU context switching, PCIe bus transfer latency, and software-based packet scheduling. When a host operating system attempts to classify millions of packets per second, cache thrashing and interrupt handling cause catastrophic system jitter and tail latency spikes¹³.

Network slicing fundamentally enables 5G operators to multiplex multiple independent, logical networks over a shared physical infrastructure⁴. A slice dedicated to autonomous vehicle telemetry (Ultra-Reliable Low-Latency Communication, or URLLC) requires microsecond latency, whereas a slice for streaming augmented reality (Enhanced Mobile Broadband, or eMBB) requires massive sustained bandwidth. If all traffic traverses the same hardware queues without isolation, eMBB burst traffic will congest the buffers, introducing lethal delays to the URLLC traffic.

Quality of Service (QoS) enforcement exists to police these slicing boundaries. QoS algorithms, such as Token Buckets, Two-Rate Three-Color Markers, and Strict Priority schedulers, dynamically regulate the rate at which packets are injected into the physical wire¹¹.

Packet processing becomes a bottleneck because identifying the slice (parsing S-NSSAI tags and GTP tunnel IDs) and calculating QoS token allowances requires stateful memory lookups for every individual packet⁴. FPGA acceleration is crucial here because hardware match-action tables and parallel queue state managers can process these lookups in deterministic nanoseconds, completely bypassing the host CPU and PCIe bus¹¹.

Part 3: Deep Literature Survey

The architecture proposed must be grounded in the vanguard of current academic research. The following analyses deconstruct the most critical literature shaping FPGA SmartNICs and network slicing.

Research Paper	Corundum: An Open-Source 100-Gbps
-----------------------	--

	NIC
Problem	Absence of an open-source, high-performance, and extensible NIC platform capable of realistic 100Gbps line rates for networking research.
Architecture	A modular, parameterized FPGA design storing massive queue states in Block RAM, connected via a custom segmented memory interface to a PCIe DMA engine ²⁴ .
Contribution	Support for over 10,000 individually controllable transmit and receive queues coupled with modular, hardware-level transmit schedulers ¹² .
Hardware Blocks	Custom PCIe DMA, mqnic_core, descriptor fetch modules, Round-Robin/TDMA transmit schedulers ²⁴ .
FPGA Used	Xilinx Virtex UltraScale+ (NetFPGA SUME, Alveo U50/U200/U250/U280) ¹² .
Results	Achieved 100Gbps operation with precise time-division multiple access (TDMA) scheduling and zero CPU overhead for transmission control ¹² .
Limitations	The highly custom DMA subsystem presents a steep learning curve and limits interoperability with standard vendor IPs (like Xilinx QDMA) ³¹ .
Future Work	Integration of higher-level transport offloads and more advanced, dynamic scheduling algorithms.
Research Paper	Enabling Time-Aware Priority Traffic Management over Distributed FPGA

	Nodes
Problem	The inability of standard NICs to enforce strict, time-aware bandwidth and latency guarantees for network slices in distributed environments without heavy CPU involvement.
Architecture	An extension of the Corundum SmartNIC framework. Dedicated hardware control registers are exposed on the AXI bus, mapped to the Linux QDISC mechanism ¹¹ .
Contribution	A hardware-enforced TDMA scheduler that allows a host OS to assign fractional transmission time-slots to specific queues, guaranteeing bandwidth per traffic class ¹¹ .
Hardware Blocks	Time-aware hardware schedulers, AXI configuration registers, priority mapping logic ¹¹ .
FPGA Used	AMD Alveo U50 and System-on-Chip (SoC) FPGAs ¹¹ .
Results	Demonstrated precise runtime configurability of packet scheduling, achieving near-perfect isolation of traffic classes and bandwidth allocation ¹¹ .
Limitations	The control plane logic (deciding the time slots) resides on the host CPU, introducing PCIe latency when highly dynamic, microsecond-level QoS adjustments are required ¹¹ .
Future Work	Migrating the control plane closer to the data plane to eliminate host intervention.

Research Paper	SCENIC: A SmartNIC for Flexible,
-----------------------	---

	High-Performance In-Network Compute
Problem	The hard trade-off between the high bandwidth of closed-source commercial SmartNICs and the low bandwidth/poor integration of open-source academic SmartNICs ⁹ .
Architecture	A heterogeneous architecture deploying offloaded applications in both programmable logic (FPGA) and on-chip ARM cores, backed by a virtual memory model for GPU/SSD direct access ⁹ .
Contribution	An end-to-end open-source 100G platform providing native OS integration (Linux netdev) and hardware-offloaded RDMA and TCP/IP stacks ⁹ .
Hardware Blocks	OpenNIC shell foundation, hardware RDMA/TCP engines, ARM processing subsystem interconnects ¹⁴ .
FPGA Used	AMD/Xilinx Alveo series (U250/U280).
Results	Delivered data processing offload performance comparable to commercial DPUs while maintaining an entirely open-source stack ¹⁴ .
Limitations	Relies heavily on hard ARM cores, leaving a gap for architectures that prefer lightweight, open-source RISC-V soft cores for control plane execution.
Future Work	Expanding the library of supported in-network compute applications (e.g., machine learning inference).

Research Paper	P4THLS: Templated High-Level Synthesis
-----------------------	---

	for P4 Data-Plane Applications
Problem	The extreme complexity of manually translating high-level network data plane abstractions (P4) into low-level FPGA hardware description languages ²⁰ .
Architecture	A templated High-Level Synthesis (HLS) framework that maps P4 programs into synthesizable C++ code, leveraging AXI data streams and configurable bus widths ²⁰ .
Contribution	Native support for P4 stateful objects (registers, counters, meters) and structured metadata propagation across FPGA pipeline stages without manual RTL coding ¹⁹ .
Hardware Blocks	HLS-generated parsers, Match-Action tables (HLSCAM), and deparsers ¹⁹ .
FPGA Used	AMD Alveo U280 ²⁰ .
Results	Achieved 143.3 Gbps throughput with a 512-bit bus, sustaining sub-50-cycle processing latency at 250 MHz ²⁰ .
Limitations	HLS-generated pipelines inherently consume more logic resources and may struggle with the ultimate timing closure limits compared to hand-tuned Verilog.
Future Work	Further optimization of HLS pragmas to rival hand-written RTL efficiency.
Research Paper	P4-NetFPGA-Based Network Slicing Solution for 5G MEC Architectures

Problem	The need for deterministic, hardware-enforced isolation of network slices at the mobile edge, avoiding software vSwitch bottlenecks ¹⁰ .
Architecture	A P4-defined data plane compiled onto the NetFPGA framework, utilizing custom match-action tables to identify 5G users and assign them to isolated hardware queues ¹⁰ .
Contribution	Demonstrated hardware-isolation of performance metrics (bandwidth, latency, and packet loss) for specific 5G traffic slices traversing the edge-to-core segment ¹⁰ .
Hardware Blocks	NetFPGA SUME reference pipeline, SDNet-compiled P4 parsers, priority queuing systems.
FPGA Used	NetFPGA SUME (Xilinx Virtex-7) ³² .
Results	Validated strict QoS isolation in a real 5G infrastructure, eliminating the jitter associated with software slicing ¹⁰ .
Limitations	Deployed on the older NetFPGA SUME hardware using deprecated SDNet compilers, limiting scalability to modern 100G/400G environments ³² .
Future Work	Migrating the slicing logic to modern Alveo platforms and VitisNetP4.

Project Fit Among Existing Work

The proposed "RISC-V Controlled FPGA SmartNIC for Network Slicing" perfectly bridges the gaps identified in current literature. While *Corundum* provides exceptional queueing mechanics¹², and *Time-Aware Schedulers* prove QoS efficacy¹¹, they rely on the host CPU for the control plane. While *SCENIC* offloads the control plane to hard ARM cores¹⁴, it lacks deep focus on 5G slicing QoS. By embedding a RISC-V soft processor directly on the FPGA to locally

control hardware QoS schedulers (designed in RTL), this project achieves microsecond-latency control loops for 5G network slicing, entirely independent of the host CPU.

Part 4: Repository Reverse Engineering

A robust architecture prevents the redundant engineering of foundational components. The following analyzes the critical open-source repositories available for the design space.

ZeusUDP

Question	Analysis
What does it do?	Implements a lightweight, hardware-accelerated UDP/IP network stack ⁴⁸ .
What problem does it solve?	Bypasses the need for complex, heavy operating system networking stacks for simple UDP streaming applications.
Architecture implemented?	A deeply pipelined, stream-based architecture that sequentially parses Ethernet, IPv4, and UDP headers ⁴⁹ .
Hardware blocks existing?	Header generators, CRC/checksum calculators, stream payload combiners/splitters ⁴⁹ .
Reusable / Not Reusable?	Reusable: The checksum and header parsing logic for UDP endpoints. Not Reusable: It lacks PCIe DMA interfaces and TCP state tracking; it cannot function as a full SmartNIC ⁴⁹ .
Assumptions made?	Assumes endpoint termination of UDP traffic, not deep packet inspection or transit routing ⁴⁹ .
Limitations exist?	Limited to UDP. Does not support SR-IOV or advanced queue management ⁴⁹ .
Difficulty to learn?	Moderate. Requires familiarity with SystemVerilog and Bluespec ⁴⁸ .

Maturity & Activity?	Moderate maturity; active within specific sub-communities ⁴⁸ .
Foundation Recommendation?	Not recommended as a foundation for a 5G SmartNIC due to lack of PCIe and complex queueing.

Summer23_SmartNIC

Question	Analysis
What does it do?	Merges a RISC-V soft core (RocketChip) with a 100G networking shell (OpenNIC) on an FPGA ²⁷ .
What problem does it solve?	Provides a proof-of-concept for embedding localized control plane processing directly inside a high-speed data plane shell ²⁷ .
Architecture implemented?	The OpenNIC shell architecture, with the RocketChip core instantiated as a custom IP inside the 250MHz user role box ²⁷ .
Hardware blocks existing?	OpenNIC (QDMA, CMAC) + RocketChip (RISC-V core, memory controllers) ²⁷ .
Reusable / Not Reusable?	Reusable: The Tcl scripts and block design methodologies for integrating RocketChip with OpenNIC ²⁷ . Not Reusable: The project lacks a fully implemented 5G dataplane or QoS engine.
Assumptions made?	Assumes the primary goal is integration validation rather than line-rate packet processing ²⁷ .
Limitations exist?	It is an internship proof-of-concept; the data path between the network and the RISC-V core is rudimentary ²⁷ .

Difficulty to learn?	Moderate to High. Requires navigating Vivado block designs, Tcl scripting, and Xilinx IP integration ²⁷ .
Maturity & Activity?	Low maturity (Academic PoC). Inactive ²⁷ .
Foundation Recommendation?	Highly recommended as an integration reference guide, but not as the primary source code foundation.

OpenNIC

Question	Analysis
What does it do?	Delivers an open-source, FPGA-based 100Gbps PCIe NIC shell for Xilinx Alveo boards ⁶ .
What problem does it solve?	Eliminates the hardware boilerplate problem by providing pre-verified PCIe DMA and 100G Ethernet MAC implementations ⁶ .
Architecture implemented?	A "Shell and Role" architecture. The shell manages the QDMA and CMAc, exposing standardized 250MHz and 322MHz AXI-Stream interfaces to the user's "role" logic boxes ⁶ .
Hardware blocks existing?	Xilinx QDMA Subsystem, CMAc Subsystem, Packet Adapters (FIFOs), and AXI-Lite System Configuration modules ⁶ .
Reusable / Not Reusable?	Reusable: The entire shell framework is designed specifically to be reused. Not Reusable: Nothing; the repository is purely foundational ⁶ .
Assumptions made?	Assumes the user will provide all layer-3/layer-4 packet processing and

	scheduling logic inside the role box ⁷ .
Limitations exist?	It is inherently <i>just</i> a shell. It lacks native network stacks, SR-IOV (without extension), and advanced queue schedulers out of the box ²⁹ .
Difficulty to learn?	Moderate. Requires a solid understanding of AXI-Stream, AXI-Lite, and Vivado Tcl build flows ⁶ .
Maturity & Activity?	Very mature (AMD/Xilinx backed) and actively maintained ²⁸ .
Foundation Recommendation?	Strongly recommended as the absolute foundation for the hardware data plane.

Corundum

Question	Analysis
What does it do?	Provides an open-source, high-performance FPGA-based NIC framework optimized for massive-scale queue management ¹² .
What problem does it solve?	Solves the limitation of fixed, inflexible queueing and scheduling in standard NICs, enabling fine-grained control over tens of thousands of queues ¹² .
Architecture implemented?	A highly parameterized architecture storing queue states in FPGA Block RAM, driven by a custom segmented memory PCIe interface and modular schedulers ²⁴ .
Hardware blocks existing?	cpl_queue_manager, desc_fetch, mqnic_core, TDMA/Round-Robin schedulers, MAC/PHY wrappers ²⁴ .

Reusable / Not Reusable?	Reusable: The conceptual designs of the schedulers and queue managers. Not Reusable: The custom DMA subsystem is extremely complex to untangle from the rest of the logic ²⁴ .
Assumptions made?	Assumes the user wants to utilize the proprietary Corundum Linux driver and custom PCIe data movers rather than standard vendor IPs ²⁵ .
Limitations exist?	Integrating standard Xilinx IPs (like Vitis HLS blocks) into Corundum's custom memory interfaces is significantly harder than using standard AXI streams ²⁵ .
Difficulty to learn?	Very High. The codebase is massive, highly parameterized, and uses non-standard DMA concepts ²⁴ .
Maturity & Activity?	Highly mature, widely published, and actively maintained ⁴⁴ .
Foundation Recommendation?	Not recommended as the base shell due to complexity, but highly recommended as a reference for designing the QoS queue managers.

RocketChip

Question	Analysis
What does it do?	An open-source SoC generator that instantiates RISC-V Rocket cores and associated memory hierarchies ⁵¹ .
What problem does it solve?	Provides a highly parameterized, silicon-proven, open-source processor ecosystem for custom SoC design ⁸ .

Architecture implemented?	A scalable Coreplex architecture featuring Rocket (or BOOM) cores, L1/L2 caches, a SystemBus, a PeripheryBus, and RoCC coprocessor interfaces ⁵¹ .
Hardware blocks existing?	CPU cores, page-table walkers, caches, UART/GPIO peripherals, and TileLink-to-AXI converters ⁵¹ .
Reusable / Not Reusable?	Reusable: The entire generated core and the PeripheryBus (AXI-Lite) for attaching custom MMIO registers ⁵¹ . Not Reusable: Modification of the internal CPU pipeline requires learning Chisel, which is generally avoided by VLSI engineers focused on networking.
Assumptions made?	Assumes the use of the Scala/Chisel hardware construction language for configuration and generation ⁸ .
Limitations exist?	Generating the Verilog requires a complex Scala toolchain. The native network interfaces (SimpleNIC) are basic and not suited for 100G ⁵⁴ .
Difficulty to learn?	Extremely High. Mastering Chisel and the "Diplomacy" parameter negotiation framework takes months ⁵³ .
Maturity & Activity?	Industry-standard maturity (Chips Alliance) and highly active ⁵¹ .
Foundation Recommendation?	Strongly recommended as the generator for the RISC-V control plane core.

Synthesis and Final Foundation Recommendation: If building the architecture from scratch, the design must utilize **OpenNIC** as the foundational 100G data plane shell, completely avoiding the task of rewriting PCIe and MAC logic⁶. The control plane must be generated using **RocketChip**⁵¹, embedded into OpenNIC's 250MHz user role box using the integration strategies demonstrated in **Summer23_SmartNIC**²⁷. The custom QoS logic should be heavily inspired by the data structures found in **Corundum**²⁴, but implemented using standard AXI

interfaces for seamless OpenNIC compatibility.

Part 5: Project Decomposition

Assuming the execution of the "RISC-V SmartNIC for 5G Network Slicing" project, the architecture decomposes conceptually into six major, interconnected modules residing entirely within the FPGA.

Module	1. CMAC Subsystem (Physical & Link Layer)
Purpose	To interface with the optical transceivers, handle IEEE 802.3 Ethernet framing, and provide reliable link status.
Inputs/Outputs	Inputs: 100GbE serial physical lanes. Outputs: 512-bit AXI-Stream interface running at 322MHz ⁶ .
Internal Responsibilities	Clock recovery, framing, CRC verification, and basic MAC filtering.
Interaction	Feeds raw Ethernet frames directly into the Packet Adapter/Parser.
Complexity & Strategy	Low complexity. Utilize the pre-configured Xilinx Hard IP provided by the OpenNIC shell ⁶ .

Module	2. QDMA Subsystem (Host Interface)
Purpose	To facilitate high-throughput, low-latency data movement between the FPGA and the host CPU memory via PCIe ²⁹ .
Inputs/Outputs	Inputs: PCIe lanes. Outputs: 512-bit AXI-Stream interface running at 250MHz ⁶ .
Internal Responsibilities	Managing scatter-gather DMA descriptor queues, handling Physical/Virtual Functions (SR-IOV), and generating host interrupts ²⁸ .

Interaction	Receives packets destined for the host OS and transmits packets originating from host applications.
Complexity & Strategy	Moderate complexity. Leverage the OpenNIC shell's wrapper around the Xilinx QDMA IP, avoiding custom DMA design ⁶ .

Module	3. 5G Packet Parser & Classifier (Dataplane)
Purpose	To perform Deep Header Inspection (DHI) on incoming traffic to identify specific 5G network slices ⁵ .
Inputs/Outputs	Inputs: Raw AXI-Stream packets. Outputs: AXI-Stream packets appended with a custom TUSER metadata header containing the detected Slice ID.
Internal Responsibilities	Parses Ethernet, IPv4/IPv6, UDP, and 5G GTP-U headers. Extracts the Tunnel Endpoint Identifier (TEID) or S-NSSAI tags. Matches these tags against a hardware lookup table to assign the Slice ID ⁴ .
Interaction	Sits inline on the datapath. Feeds classified packets to the QoS Scheduler.
Complexity & Strategy	High complexity. Implemented as a deeply pipelined Verilog state machine or generated via a P4-to-HLS compiler (e.g., P4THLS) ¹⁹ . Requires learning network protocol structures.

Module	4. QoS Traffic Manager & Scheduler (Dataplane)
---------------	---

Purpose	To enforce strict bandwidth limits and latency guarantees for different network slices, preventing "noisy neighbor" congestion ¹¹ .
Inputs/Outputs	Inputs: Classified packets with Slice ID metadata. Outputs: Scheduled packets released to the CMAC or QDMA.
Internal Responsibilities	Maintains Token Bucket meters for each slice. Buffers packets in Block RAM if they exceed allowed rates. Employs a Strict Priority or Deficit Round Robin scheduler to multiplex packets back onto the outgoing link based on slice SLA ¹¹ .
Interaction	Reads configuration registers (Token Rates, Queue Priorities) written dynamically by the RISC-V core.
Complexity & Strategy	Very high complexity. Requires mastering SRAM/BRAM memory management and parallel state tracking (inspired by Corundum's queue manager) ²⁴ .

Module	5. RISC-V Control Core (Control Plane)
Purpose	To act as the localized, autonomous "brain" of the SmartNIC, executing 5G Control Plane directives (e.g., SMF policy updates) in real-time ⁹ .
Inputs/Outputs	Inputs: Telemetry data from the datapath; API commands from the host. Outputs: AXI-Lite memory-mapped register writes.
Internal Responsibilities	Runs a lightweight RTOS or bare-metal C code. Calculates required Token Bucket refresh rates based on 5G SLA parameters and writes these values into the QoS Traffic

	Manager registers ⁹ .
Interaction	Connects to the host via a PCIe mailbox and controls the datapath modules via the AXI-Lite PeripheryBus ⁵¹ .
Complexity & Strategy	Very high complexity. Generate the core Verilog using RocketChip, instantiate it in the OpenNIC 250MHz box, and compile C firmware using the RISC-V GNU Toolchain ⁸ .

Module	6. AXI Interconnect & Synchronization
Purpose	To safely bridge multiple clock domains and multiplex memory-mapped register access across the FPGA fabric ⁶ .
Inputs/Outputs	AXI-Lite and AXI-Stream buses spanning 125MHz, 250MHz, and 322MHz domains ⁶ .
Internal Responsibilities	Clock Domain Crossing (CDC) using asynchronous FIFOs; address decoding for the RISC-V MMIO space.
Interaction	Forms the communication backbone of the entire SmartNIC.
Complexity & Strategy	Moderate complexity. Utilize standard Vivado AXI Interconnect and AXI SmartConnect IPs.

Part 6: Identifying the Real Contribution

A common pitfall in graduate-level systems research is expending months engineering foundational infrastructure that already exists, leaving insufficient time for the novel contribution. To ensure publishability and high industry relevance, the project must be strictly compartmentalized.

Category A: Existing Infrastructure (Reuse Entirely)

This category encompasses components where writing custom RTL yields zero academic or career value.

- **The 100G Ethernet MAC (CMAC) and PCIe Gen3/Gen4 DMA (QDMA):** Reuse the OpenNIC shell implementations. Designing a custom DMA is a massive undertaking that distracts from networking logic⁶.
- **The RISC-V CPU RTL:** Use the Verilog emitted by the RocketChip generator⁵¹. Do not attempt to write a custom RISC-V pipeline from scratch.
- **Standard Interconnects:** Reuse Vivado's AXI SmartConnect and asynchronous FIFO IPs for clock domain crossing⁶.

Category B: Existing Open-Source Components (Modify)

This category involves adapting existing open-source codebases to fit the specific architecture.

- **OpenNIC User Role Wrapper:** Must be modified to intercept the AXI-Stream datapath and route a subset of control traffic (e.g., specific UDP ports) directly to the RISC-V core's memory space, rather than forwarding it to the host CPU⁶.
- **RocketChip PeripheryBus Map:** Must be extended in the Scala/Chisel configuration to define the specific memory addresses that will map to the custom QoS registers⁵¹.

Category C: Components to Design from Scratch (The Core Contribution)

This category represents the actual VLSI and research contribution of the project.

- **The 5G Hardware Slicer (RTL):** Designing a high-speed, parallel match-action pipeline that identifies 5G GTP-U/S-NSSAI traffic flows at line rate. Operating at 250MHz for 100Gbps requires processing 400 bits per clock cycle, demanding aggressive pipelining and fan-out reduction⁵.
- **The QoS Enforcer (RTL):** Designing a novel, stateful queueing architecture. Implementing a hardware Token Bucket algorithm capable of updating token counts and scheduling decisions across hundreds of slice queues within a single clock cycle is a massive VLSI challenge¹¹.
- **The RISC-V Slicing Firmware (C/C++):** Writing the localized control-plane application that translates high-level 5G slice requirements (e.g., "URLLC Slice 1 requires 1ms max latency") into the specific hardware register hex values required by the QoS Enforcer⁹.

Publishable Contribution vs. Simple Integration

Simply instantiating RocketChip inside OpenNIC is an *integration exercise*, not a research publication. A publishable contribution requires empirical proof that the *system* solves a systemic bottleneck. The publication narrative must demonstrate that moving the 5G QoS scheduling logic from a software virtual switch (like OVS) into the custom FPGA logic, managed locally by the RISC-V core, reduces UPF tail latency by an order of magnitude under heavy traffic loads³⁸.

Part 7: Research Directions Evaluation

The researcher has multiple potential trajectories within the FPGA networking domain. The

following evaluates and ranks these directions based on seven critical criteria.

Rank	Project Direction	Novelty	Difficulty	VLSI Relevance	FPGA Relevance	Bharat5G Relevance	Publicability	Industrial Relevance
1	RISC-V Controlled Smart NIC for 5G Slicing	High. Co-locating the control plane (RISC-V) with a custom hardware slicer is cutting-edge ⁹ .	High. Requires bridging Chisel CPU generation with complex Verilog networking datapaths.	Very High. Mastery of SoC design, hardware-software boundary, and AXI.	Very High. Pushes the limits of FPGA routing and timing closure at 250MHz.	Very High. Directly solves 5G UPF bottlenecks for rural/edge deployments ² .	High. Bridges computer architecture and telecommunications.	Very High. Highly marketable skill set for DPU/IPU design roles.
2	QoS Scheduling Accelerator	Medium. Hardware QoS is studied, but novel, scalable algorithms are constantly	Very High. Complex state machines and SRAM memory management require	Very High. Pure, aggressive RTL design.	Very High. Excellent demonstration of FPGA resource management (BRAM/URA	High. Relevant to all edge and core network management.	High. Fits well into networking conferences (SIGCOMM, NSDI).	High. Deeply relevant to cloud infrastructure and ASIC networking.

		needed ¹¹ .	ed to meet timing.		M).			
3	Network Slicing Accelerator	Medium. Parsing headers in hardware is well understood (e.g., P4 pipelines) ¹⁰ .	Medium. Writing parsers in Verilog is straightforward state machine design.	Medium. Good RTL practice, but lacks full-system SoC complexity.	High. Standard FPGA networking workload.	High. Fits standard 5G Core offloaded narratives.	Medium. Needs a highly novel angle to stand out from existing P4 literature.	High. Relevant for standard Smart NIC development.
4	AI-assisted Smart NIC	Very High. Running neural networks on network flows inline for security or predictive QoS ⁹ .	Extreme. Requires HLS, ML accelerators, and strict timing constraints.	High. Excellent for AI-hardware engineering roles.	Very High. Showcases advanced DSP and matrix multiplication on FPGA.	Medium. Highly academic, but further from immediate telecom deployment.	Very High. AI-networking is the hottest current academic topic.	Medium. Industry is still figuring out use cases for inline AI.
5	Open RAN	Low. Heavily	High. Requires	High. Deep	High. Excellent	High. Relevant	Low. Hard	High. Telecom

	Smart NIC (eCPR I offload)	y commercialized by NVIDIA and Intel ³⁵ .	es nanoscond-level PTP timing logic and heavy DSP ³⁵ .	DSP and timing closure.	ent use of FPGA transceivers.	nt to telecommunications hardware.	to publishing against massive corporate R&D budgets.	m equipment manufacturers heavily recruit for this.
6	UAV / Drone Communication Smart NIC	High. Architectures constrained by Size, Weight, and Power (SWaP) ³⁶ .	Medium. Can use smaller, simpler boards (Zynq) rather than massive Alveos.	Medium. Good embedded systems practice.	Medium. Less focus on massive throughput.	Low. A niche application compared to core data center edge nodes.	Medium. Fits well into specialized IoT or drone conferences.	Medium. Niche aerospace/defense relevance.
7	ZeusUDP Extension	Low. UDP offload is a fundamentally solved problem ⁴⁸ .	Low. Basic Verilog state machines.	Low. Insufficient complexity for a graduate-level VLSI	Low. Does not utilize advanced FPGA features.	Low. Does not address modern 5G virtualized worklo	Low. Unlikely to be accepted in major venues.	Low. Too simplistic for modern datacenter roles.

				projec t.		ads.		
--	--	--	--	--------------	--	------	--	--

Justification for Ranking: The "RISC-V Controlled SmartNIC for 5G Slicing" is ranked first because it provides the ultimate balance. It is difficult enough to prove elite VLSI competency, novel enough to secure high-tier publications, and perfectly aligned with Bharat5G Labs' mandate to innovate in 5G network infrastructure².

Part 8: Learning Roadmap

Assuming existing foundational knowledge (Verilog, Digital Design, Basic FPGA, Basic AXI, Basic Computer Architecture), the specific knowledge gaps preventing immediate execution are: SoC Integration methodologies, PCIe/DMA system architectures, High-Speed Networking (100G) clock domains, and 5G Core Protocol structures.

Phase 1: Advanced Interconnects & PCIe (Weeks 1-3)

- **Gap:** Moving from "Basic AXI" to handling AXI4-Stream (TVALID, TREADY, TLAST, TKEEP) at 512-bit widths, and understanding how PCIe translates host memory to hardware via DMA.
- **Documentation:** Xilinx PG302 (QDMA Subsystem for PCIe) Product Guide. Focus heavily on the streaming interfaces, queue structures, and descriptor bypassing²⁸.
- **Repository:** Clone OpenNIC. Run the Vivado block design simulations focusing on the QDMA loopback tests to visualize data movement⁶.

Phase 2: 5G Core and Network Slicing (Weeks 4-5)

- **Gap:** Understanding exactly what a SmartNIC is supposed to process. What is the byte structure of a GTP-U packet? What is an S-NSSAI tag?
- **Papers:** "P4-NetFPGA-based network slicing solution for 5G MEC architectures"¹⁰. "Network Slicing Technology" (NEC Technical Journal)⁴.
- **Standards:** Read 3GPP TS 23.501 (System architecture for the 5G System), specifically Sections 5.7 (QoS model) and 5.8 (User Plane management).

Phase 3: The RocketChip & RISC-V Ecosystem (Weeks 6-8)

- **Gap:** Generating a soft-core SoC. Understanding Chisel is necessary not to write the CPU core, but to configure the generator parameters to emit the correct Verilog.
- **Documentation:** The official Chipyard/RocketChip documentation. Focus strictly on the "MMIO Peripherals" and the PeripheryBus sections⁵¹.
- **Repository:** Deeply study the Summer23_SmartNIC repository. Analyze the block design files (.bd) to see exactly how the AXI lines from the RocketChip wrapper connect to the surrounding Vivado logic²⁷.

Phase 4: Hardware QoS and Scheduling (Weeks 9-11)

- **Gap:** Designing scalable data structures in hardware to hold packet pointers and token

bucket counts without violating timing constraints.

- **Papers:** "Corundum: An Open-Source 100-Gbps NIC"¹². "Enabling Time-Aware Priority Traffic Management over Distributed FPGA Nodes"¹¹.
- **Repository:** Navigate the Corundum repository. Isolate the `mqnic_tx_scheduler_block` and `queue_manager` Verilog modules to study how they manage state transitions in Block RAM²⁴.

Part 9: Final Recommendation and Execution Strategy

The conclusive recommendation is to commit to the **RISC-V Controlled SmartNIC for 5G Network Slicing**. It offers the highest synthesis of VLSI engineering, networking, and academic novelty.

Architectural Blueprint

- **Foundation:** Utilize the **OpenNIC** shell deployed on a Xilinx Alveo (e.g., U50 or U280). This abstracts away the immense complexities of 100G Ethernet MACs and PCIe Gen4²⁸.
- **Control Plane:** Instantiate a single-core **RocketChip** (generated via Chipyard) inside the 250MHz user role box of OpenNIC⁶.
- **Data Plane (The Core Work):** Write custom Verilog for a **5G Slicer and QoS Scheduler**. Place this inline on the 512-bit AXI-Stream datapath between the CMAC and QDMA. Connect its configuration registers to the RocketChip via an AXI-Lite interconnect⁶.

Minimum Viable Project (MVP)

1. Instantiate the OpenNIC shell and achieve successful synthesis.
2. Build a hardcoded Verilog packet parser that reads an incoming IPv4/UDP packet and sorts it into two basic queues (High Priority and Low Priority).
3. Implement a simple Strict Priority hardware scheduler that drains the High Priority queue before the Low Priority queue.
4. Prove via simulation that under heavy traffic, High Priority packets experience lower latency.

Strong VLSI Resume Version

1. Expand the MVP by implementing a dynamic Token Bucket rate-limiter for each hardware queue.
2. Integrate the RocketChip core alongside the datapath.
3. Write bare-metal C code for the RISC-V core that allows a user to update the Token Bucket rates in real-time via a PCIe mailbox interface⁹.
4. Achieve successful timing closure at 250MHz for the entire 512-bit datapath. This proves to industry recruiters an elite understanding of pipeline stages, fan-out reduction, and synchronous digital design.

Strong Bharat5G / Publication Version

1. Complete the VLSI version, but specifically tune the packet parser to decode 5G GTP-U

headers and 3GPP S-NSSAI slicing tags⁴.

2. Set up a physical testbed with two servers. Run an open-source 5G Core (like Open5GS) with a simulated Multi-Access Edge Computing (MEC) workload—for example, a high-bandwidth eMBB video stream and a latency-sensitive URLLC drone control stream³⁴.
3. Measure and compare the tail latency of the URLLC stream when routed through a standard software UPF (OVS) versus the custom hardware-accelerated UPF.
4. The resulting empirical data—proving deterministic, microsecond latency isolation for critical 5G slices at 100Gbps—will form the core of a highly publishable paper, fulfilling the mandate of Bharat5G Labs and cementing the researcher's expertise in the field².

Works cited

1. About Us - Bharat 5G Labs, <https://www.bharat5glabs.gov.in/page/about-us>
2. Institution - Bharat 5G Labs, <https://www.bharat5glabs.gov.in/page/institution>
3. 5G applications - Bharat 5G Labs, <https://www.bharat5glabs.gov.in/page/5g-applications>
4. NEC's Network Slicing Supports People's Lives in the 5G Era, <https://www.nec.com/en/global/techrep/journal/g23/n01/230122.html>
5. Packet Classification in 5G UPF (User Plane Function) Fixing LPM Routing Bottlenecks in New 5G Networks - MoSys, Inc., <https://www2.perasoinc.com/applications/packet-classification-in-5g-upf-user-plane-function-fixing-lpm-routing-bottlenecks-in-new-5g-networks/>
6. GitHub - Xilinx/open-nic-shell: AMD OpenNIC Shell includes the HDL source files, <https://github.com/Xilinx/open-nic-shell>
7. MicroZed Chronicles: Alveo Edition OpenNIC - Aduvo Engineering, <https://www.aduvoengineering.com/post/microzed-chronicles-alveo-edition-opennic>
8. GaloisInc/BESSPIN-rocket-chip: The BESSPIN fork of the Rocket chip generator. - GitHub, <https://github.com/GaloisInc/BESSPIN-rocket-chip>
9. SCENIC: Stream Computation-Enhanced SmartNIC - arXiv, <https://arxiv.org/html/2604.15128v1>
10. P4-NetFPGA-based network slicing solution for 5G MEC architectures - IEEE Xplore, <https://ieeexplore.ieee.org/document/8901889/>
11. Enabling Time-Aware Priority Traffic Management over Distributed FPGA Nodes - arXiv, <https://arxiv.org/pdf/2509.26043>
12. Corundum: An Open-Source 100-Gbps NIC - University of California San Diego, <https://cseweb.ucsd.edu/~snoeren/papers/corundum-fccm20.pdf>
13. Solution Brief - Intel Infrastructure Processing Units (IPUs) Leverage Napatech's Virtualized Data Plane Software to Enable Breakthrough Performance for Microservices-based Cloud Applications, <https://www.intel.com/content/dam/www/central-libraries/us/en/documents/2023-07/ipu-for-microservices-solution-brief.pdf>
14. SCENIC: Stream Computation-Enhanced SmartNIC - arXiv,

- <https://arxiv.org/pdf/2604.15128>
15. Reconfigurable SmartNICs: A Comprehensive Review of FPGA Shells and Heterogeneous Offloading Architectures - MDPI, <https://www.mdpi.com/2076-3417/16/3/1476>
 16. SPAC: Automating FPGA-based Network Switches with Protocol Adaptive Customization, <https://arxiv.org/html/2604.21881v1>
 17. Intel Mount Evans IPU Overview | PDF | Virtual Machine | Cloud Computing - Scribd, <https://www.scribd.com/document/522127863/Intel-Hotchips-2021-Mt-Evans-R2a>
 18. Intel's Hyperscale-Ready Infrastructure - Processing Unit (IPU) - HotChips 33, <https://hc33.hotchips.org/assets/program/conference/day1/Intel%20TLM%20Hotchips%202021%20-%20Mt%20Evans%20R2a%20-%20Final%20version%202.pdf>
 19. A Stateful Extension to P4THLS for Advanced Telemetry and Flow Control - MDPI, <https://www.mdpi.com/1999-5903/17/11/530>
 20. P4THLS: A Templated HLS Framework to Automate Efficient Mapping of P4 Data-Plane Applications to FPGAs - IEEE Xplore, <https://ieeexplore.ieee.org/iel8/6287639/10820123/11168164.pdf>
 21. Real-Time In-Network Machine Learning on P4-Programmable FPGA SmartNICs with Fixed-Point Arithmetic and Taylor Approximations - arXiv, <https://arxiv.org/html/2507.00428v1>
 22. A RISC-V in-network accelerator for flexible high-performance low-power packet processing - arXiv, <https://arxiv.org/pdf/2010.03536>
 23. OpenNIC FPGA SmartNIC Shell on iW-Fibre Platforms - iWave Global, <https://iwave-global.com/articles/opennic-fpga-smartnic-shell-on-iw-fibre-platforms/>
 24. GitHub - corundum/corundum: Open source FPGA-based NIC and platform for in-network compute, <https://github.com/corundum/corundum>
 25. 8.1. Overview - Corundum documentation, <https://docs.corundum.io/en/latest/modules/overview.html>
 26. Re: About corundum - Google Groups, <https://groups.google.com/g/corundum-nic/c/bV2BtcaGBGk>
 27. MrJimbo2002/Summer23_SmartNIC: A repository for summer internship designing a SmartNIC combining RISC-V and NetFPGA - GitHub, https://github.com/MrJimbo2002/Summer23_SmartNIC
 28. README.md · 6cb924b03c01b1a6dc94be9e541eec28cecd335 · Mario Vega / onspfft - GitLab, <https://socks.lbl.gov/mvega/onspfft/-/blob/6cb924b03c01b1a6dc94be9e541eec28cecd335/README.md>
 29. Utilizing OpenNIC, DPDK and SR-IOV on FB4XXVG@Z21D, https://www.silicom.dk/wp-content/uploads/2023/11/SB_ONIC_SRIOV_DPDK.pdf
 30. ESnet SmartNIC hardware design repository. - GitHub, <https://github.com/esnet/esnet-smartnic-hw>
 31. open-nic/FAQ.md at main · Xilinx/open-nic - GitHub, <https://github.com/Xilinx/open-nic/blob/main/FAQ.md>
 32. The P4->NetFPGA Workflow for Line-Rate Packet Processing - Apollo,

- <https://www.repository.cam.ac.uk/items/5a99c7c3-b959-4c83-a7ef-5f5f01fe3c2c>
33. NetFPGA SUME: Toward 100 Gbps as Research Commodity - Department of Engineering Science, <https://eng.ox.ac.uk/media/5163/zilberman2014sume.pdf>
 34. Performance Characterization of dApps in Open Radio Access Networks - arXiv, <https://arxiv.org/pdf/2605.05426>
 35. Building an Accelerated 5G CloudRAN at the Edge | NVIDIA Technical Blog, <https://developer.nvidia.com/blog/building-accelerated-5g-cloudran-at-the-edge/>
 36. AtlasRAN: Modeling and Performance Evaluation of Open 5G Platforms for Ubiquitous Wireless Networks This material is based upon work supported by the National Science Foundation under Grant Numbers CNS-2202972, CNS- 2318726, and CNS-2232048. - arXiv, <https://arxiv.org/html/2603.14661v1>
 37. 6G Core Routing with P4 and FPGA Acceleration [2026] - Tech Bytes, <https://techbytes.app/posts/6g-core-routing-p4-fpga-acceleration-deep-dive/>
 38. An eBPF-XDP hardware-based network slicing architecture for future 6G front, https://myresearchspace.uws.ac.uk/ws/portalfiles/portal/45057338/2023_10_27_Salva_Garcia_et_al_eBPF-XDP_accepted.pdf
 39. Developed 5G network slicing architecture based on a NetFPGA framework.... - ResearchGate, https://www.researchgate.net/figure/Developed-5G-network-slicing-architecture-based-on-a-NetFPGA-framework-DMA-direct_fig5_335451815
 40. P4-based Programmable Packet Processing for Custom Workloads - iWave Global, <https://iwave-global.com/articles/p4-based-programmable-packet-processing-for-custom-workloads/>
 41. NEC's Network Slicing Supports People's Lives in the 5G Era, <https://www.nec.com/en/global/techrep/journal/q23/n01/pdf/230122.pdf>
 42. An Intelligent and Programmable Data Plane for QoS-Aware Packet Processing - IEEE Xplore, <https://ieeexplore.ieee.org/iel8/9882533/10356147/10706883.pdf>
 43. An FPGA-Based High-Performance Stateful Packet Processing Method - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC10673001/>
 44. 1. Introduction — Corundum documentation, <https://docs.corundum.io/>
 45. Enabling Time-Aware Priority Traffic Management over Distributed FPGA Nodes - arXiv, <https://arxiv.org/html/2509.26043v1>
 46. Demo: Programmable Packet Scheduling on FPGA-Based Switches - IEEE Xplore, <https://ieeexplore.ieee.org/abstract/document/11314135/>
 47. FPGA-Friendly Programmable Data Plane: A Transition from P4 Semantics to Pipeline Implementations via High-Level Synthesis Approaches - PolyPublie, https://publications.polymtl.ca/71101/1/2025_MostafaAbbasMollaei.pdf
 48. cocotb · GitHub Topics, <https://github.com/topics/cocotb?l=systemverilog&o=asc&s=updated>
 49. datenlord/blue-udp: The hardware implementation of UDP in Bluespec SystemVerilog - GitHub, <https://github.com/datenlord/blue-udp>
 50. Yuzhe Jin MrJimbo2002 - GitHub, <https://github.com/MrJimbo2002>
 51. 3.1. Rocket Chip - Chipyard's documentation - Read the Docs,

- <https://chipyard.readthedocs.io/en/latest/Generators/Rocket-Chip.html>
52. 3.1. Rocket Chip — Chipyard 1.11.0 documentation,
<https://chipyard.readthedocs.io/en/stable/Generators/Rocket-Chip.html>
 53. chipsalliance/rocket-chip: Rocket Chip Generator - GitHub,
<https://github.com/chipsalliance/rocket-chip>
 54. GitHub - firesim/firechip: Top-Level Project for Firebox SoC, consisting of Rocket, BOOM, and peripherals (e.g. Ethernet NIC). This is the default target generator used in FireSim., <https://github.com/firesim/firechip>